

TREE 1

PROBLEM 3

QUESTION:

DS	Session	Tree 1	Question Information	Level 1	Challenge 63
Question description You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates $(1, 1)$, and the lower-right square has coordinates (n, n). Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest? Constraints • $1 \leq n \leq 1000$ • $1 \leq q \leq 2 \cdot 10^5$ • $1 \leq y1 \leq y2 \leq n$ • $1 \leq x1 \leq x2 \leq n$ Input The first input line has two integers n and q: the size of the forest and the number of queries. Then, there are n lines describing the forest. Each line has nn characters: $.$ is an empty square and $*$ is a tree. Finally, there are q lines describing the queries. Each line has four integers $y1, x1, y2, x2$ corresponding to the corners of a rectangle. Output Print the number of trees inside each rectangle.					
Logical Test Cases Test Case 1 INPUT (STDIN) <pre>4 3 .*.. *.* **.. **** 2 2 3 4 3 1 3 1 1 1 2 2</pre> EXPECTED OUTPUT <pre>3 1 2</pre> Test Case 2 INPUT (STDIN) <pre>4 3 .*.* *.* **.* ***. 1 2 3 4 3 2 3 4 1 4 2 3</pre> EXPECTED OUTPUT <pre>6 2 0</pre>					
Mandatory Test Cases Test Case 1 KEYWORD <pre>for(i=1;i<=n;i++)</pre>					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY <pre>8</pre> Test Case 2 TOKEN COUNT <pre>310</pre> Test Case 3 NLOC <pre>28</pre>					

SOURCE CODE:

```
#include <stdio.h>

int grid_sum[1005][1005];

int main() {
    int dimension, queries;
    if (scanf("%d %d", &dimension, &queries) != 2) return 0;

    for (int r = 1; r <= dimension; r++) {
        char line[1005];
        scanf("%s", line);
        for (int c = 1; c <= dimension; c++) {
            int has_tree = (line[c - 1] == '*') ? 1 : 0;
            grid_sum[r][c] = has_tree
                + grid_sum[r - 1][c]
                + grid_sum[r][c - 1]
                - grid_sum[r - 1][c - 1];
        }
    }

    while (queries--) {
        int r1, c1, r2, c2;
        scanf("%d %d %d %d", &r1, &c1, &r2, &c2);
        int answer = grid_sum[r2][c2]
            - grid_sum[r1 - 1][c2]
            - grid_sum[r2][c1 - 1]
            + grid_sum[r1 - 1][c1 - 1];
        printf("%d\n", answer);
    }
    return 0;
}
```

OUTPUT:

```
4 3
. * .
* . **
** . *
***
2 2 3 4
3 1 3 1
1 1 2 2
3
1
2
```

```
4 3
. * . *
* . **
** . *
*** .
1 2 3 4
3 2 3 4
1 4 2 3
6
2
0
```